

Setup Guide — every environment, every command

Written for: you, setting this project up from nothing on a new machine, or re-doing one part. Every command is spelled out. This covers the **server** (local + AWS), the **iOS toolchain** (WSL + xtool, no Mac), **signing/sideload**ing, and the **day-to-day workflows**.

Companion to [DEVELOPER GUIDE.md](#) (concepts), [SERVER INTERNALS.md](#) and [CLIENT INTERNALS.md](#) (code).

Conventions: `PS>` = Windows PowerShell, `wsl$` = inside WSL Ubuntu (login shell), `ec2$` = on the EC2 server over SSH. Paths use this machine's layout (`C:\dev\LocationTracker`).

0. What you need installed

Tool	Where	Why
Docker Desktop	Windows	Runs the server stack locally
Git	Windows	Version control
WSL 2 + Ubuntu	Windows	Hosts the Swift/xtool toolchain
Swift 6 toolchain (swiftly)	WSL	Compiles the app
xtool	WSL	Builds an iOS <code>.ipa</code> without Xcode
Darwin Swift SDK	WSL (via xtool)	The iOS SDK, installed from an <code>Xcode.xip</code>

AWS CLI v2	Windows	Creates/manages the EC2 server
Sideloadly (+ Apple iTunes drivers)	Windows	Signs and installs the .ipa on the iPhone
.NET 9 SDK	Windows (optional)	Only to run/build the server outside Docker or make EF migrations

1. Server — local development

1.1 First run

```
# from C:\dev\LocationTracker (Git Bash or WSL)
cp .env.example .env           # then edit EVERY value (se
./generate-certs.sh           # self-signed TLS cert for
docker compose up -d --build   # build + start db, api, ng
./smoke-test.sh               # 31-check end-to-end verif
```

Server is at <https://localhost> (self-signed, so `curl -k`). Postgres is not published to the host. Data lives in the `pgdata` Docker volume and survives restarts.

1.2 `.env` values (all required)

```
POSTGRES_DB=locationtracker
POSTGRES_USER=locationtracker
POSTGRES_PASSWORD=<strong random>
JWT_SIGNING_KEY=<openssl rand -base64 48>    # <32 bytes a
JWT_ISSUER=LocationTracker
JWT_AUDIENCE=LocationTracker
SEED_ADMIN_EMAIL=admin@example.com
SEED_ADMIN_PASSWORD=<12+ chars, upper/lower/digit/symbol>
```

```
CORS_ORIGIN=https://localhost
ASPNETCORE_ENVIRONMENT=Production
SWAGGER_ENABLED=false           # true to serve /swagger out
```

1.3 Everyday commands

```
docker compose ps                # status
docker compose logs -f api       # follow API logs
docker compose up -d --build     # rebuild after c
docker compose restart api       # e.g. to reset i
docker compose down              # stop (keeps dat
docker compose down -v           # stop AND delete
```

1.4 Turning Swagger on

Set `SWAGGER_ENABLED=true` in `.env`, `docker compose up -d`, open `https://localhost/swagger`. Log in via `POST /api/auth/login` first; the UI then sends the CSRF header automatically.

1.5 Database migrations (schema changes)

Needs the .NET 9 SDK + `dotnet-ef`:

```
dotnet tool install --global dotnet-ef           # once
cd src/LocationTracker.Api
dotnet ef migrations add <DescriptiveName>       # after
```

Migrations apply automatically at API startup

(`db.Database.MigrateAsync()` in `Program.cs`), so just rebuild the container. For multi-replica deployments, run migrations as a separate step instead.

2. iOS toolchain — WSL + xtool (one-time)

This is how you build an iOS app on Windows with no Mac.

2.1 WSL + Ubuntu

```
PS> wsl --install -d Ubuntu          # reboot if prompted; see
PS> wsl -l -v                       # confirm Ubuntu is vers
```

2.2 Swift toolchain (inside WSL)

```
wsl$ curl -O https://download.swift.org/swiftly/linux/swi
wsl$ tar xzf swiftly-$(uname -m).tar.gz && ./swiftly init
wsl$ . "${SWIFTLY_HOME_DIR:-$HOME/.local/share/swiftly}/e
wsl$ swift --version                # expect Swift 6.x
```

swiftly adds Swift to PATH via your shell profile — which is why xtool must always be run in a **login shell** (`bash -lc`), or it reports "Failed to obtain Swift version".

2.3 xtool

```
wsl$ curl -fL "https://github.com/xtool-org/xtool/releases
wsl$ chmod +x xtool && sudo mv xtool /usr/local/bin/
wsl$ xtool --help
```

2.4 The Darwin (iOS) SDK

xtool needs Apple's SDK, extracted from an `Xcode.xip` you supply (download from Apple, place on the Windows Desktop):

```
wsl$ xtool sdk install /mnt/c/Users/<you>/Desktop/Xcode_2
```

```
wsl$ xtool setup
```

```
# verifies the toolchain + SD
```

2.5 Project layout (already in the repo)

```
ios/
├─ Package.swift           # one library product = the app
├─ xtool.yml               # bundleID, infoPath, iconPath
├─ Info.plist              # background modes, permission
├─ Resources/AppIcon.png   # 1024×1024 icon
├─ Sources/LocationTrackerClient/*.swift
├─ package.sh              # build + wrap into .ipa on the host
└─ mkipa.py                # zips the .app into a .ipa (Windows)
```

2.6 Build the app

```
wsl$ cd /mnt/c/dev/LocationTracker/ios && ./package.sh
# runs: xtool dev build → python3 mkipa.py → copies L
```

Or from PowerShell in one line:

```
PS> wsl -d Ubuntu -- bash -lc 'cd /mnt/c/dev/LocationTracker/ios && ./package.sh'
```

Clean build $\approx$ 1 min; incremental $\approx$ 10–20 s. **Always the login shell (–1c).**

2.7 Editing settings vs code

- **Bundle ID** → `ios/xtool.yml`
 - **Background modes / permission strings / ATS** → `ios/Info.plist`
 - **Icon** → replace `ios/Resources/AppIcon.png` (1024×1024)
 - **Timers/thresholds/server URL/pinned certs** → `ios/Sources/LocationTrackerClient/AppConfig.swift`
-

3. Signing and installing on the iPhone (Sideloadly)

xtool produces an **unsigned** `.ipa`; you sign it with your Apple ID at install time.

3.1 One-time

1. Install **iTunes** (Apple's version, for USB drivers) and **Sideloadly** (sideloadly.io).
2. iPhone: **Settings** → **Privacy & Security** → **Developer Mode** → **On** (reboot when asked).

3.2 Each install

1. Plug the iPhone in over USB; tap **Trust** on the phone.
2. Sideloadly → drag in `LocationTrackerClient.ipa` → enter your Apple ID (use an app-specific password from appleid.apple.com if the account has 2FA) → **Start**.
3. iPhone: **Settings** → **General** → **VPN & Device Management** → **[your Apple ID]** → **Trust**.
4. Open the app; grant **Location** → **Always + Precise**, **Motion & Fitness**, **Notifications**.

3.3 The 7-day expiry

A free Apple ID signs for **7 days**; after that the app won't launch and tracking stops. Options: re-run Sideloadly weekly, use **AltStore/SideStore** (auto re-signs over Wi-Fi), or a **paid Apple Developer account** (\$99/yr → 1-year signatures). See [FEATURES AND LIMITS.md](#).

3.4 Shortcut automations (created on the phone, cannot

be scripted)

- **Shortcuts** → **Automation** → **+** → **Charger** → **Is Connected** → **Run Immediately** → **Open App** → **Location Tracker**. Reopens the app after a reboot (you plug in).
- Optionally **Time of Day** → **8:00 AM** → **Daily** → **Open App** as a safety net.

3.5 After changing the server certificate

The app pins the cert fingerprint. If you re-run `generate-certs.sh`, update the matching entry in

`AppConfig.pinnedCertificateSHA256` and rebuild:

```
wsl$ openssl x509 -in /mnt/c/dev/LocationTracker/nginx/cert
```

4. Server — AWS EC2 deployment

4.1 Connect the AWS CLI (once)

```
PS> winget install -e --id Amazon.AWSCLI # then open
PS> aws login # browser
# or: aws configure (Access key ID, Secret, region e
PS> aws sts get-caller-identity # confirm
```

4.2 Create the infrastructure (what was done for the live server)

Key pair, security group (SSH from your IP only; 80/443 public), instance, Elastic IP:

```

PS> aws ec2 create-key-pair --key-name reem-key --key-type rsa
PS> aws ec2 create-security-group --group-name reem-sg --protocol tcp --vpc-id <vpc-id>
PS> aws ec2 authorize-security-group-ingress --group-id <sg-id> --protocol tcp --ports 22 --vpc-id <vpc-id>
    "IpProtocol=tcp,FromPort=22,ToPort=22,IpRanges=[{CidrIp=10.0.0.0/16}]"
    "IpProtocol=tcp,FromPort=80,ToPort=80,IpRanges=[{CidrIp=10.0.0.0/16}]"
    "IpProtocol=tcp,FromPort=443,ToPort=443,IpRanges=[{CidrIp=10.0.0.0/16}]"
PS> aws ec2 run-instances --image-id <ubuntu-24.04-ami> --security-group-ids <sg> --metadata-options "HttpTokens=required" --block-device-mappings '[{"DeviceName":"/dev/sda1","Ebs":{"VolumeSize":100}}]'
PS> aws ec2 allocate-address --domain vpc
PS> aws ec2 associate-address --instance-id <id> --allocation-id <allocation-id>

```

The live server: `reem-server`, `t3.micro`, Elastic IP `34.199.20.93`, in `us-east-1`.

4.3 Provision and start (on the instance)

Copy the repo up (never `.env`, never certs — they're generated on the box):

```

# from the repo root on Windows/WSL:
git ls-files -co --exclude-standard | grep -v '^ios/' | tr '\n' ' '
| ssh -i ~/.ssh/reem-key.pem ubuntu@34.199.20.93 'mkdir -p /tmp/repo && git clone https://github.com:reem/reem.git'

```

Then on the server:

```

ec2$ cd ~/LocationTracker
ec2$ PUBLIC_IP=34.199.20.93 ./deploy/ec2-setup.sh # ins
ec2$ sudo docker compose up -d --build

```

`ec2-setup.sh` generates its own DB password, JWT key, and admin password (printed once), sets Swagger off, and issues a cert naming the public IP. Add that cert's fingerprint to the app.

4.4 Redeploy code later

```
git ls-files -co --exclude-standard | grep -v '^ios/' | t
| ssh -i ~/.ssh/reem-key.pem ubuntu@34.199.20.93 'tar -
ssh -i ~/.ssh/reem-key.pem ubuntu@34.199.20.93 'cd ~/Loca
```

On a t3.micro the .NET build is slow on standard CPU credits — flip to unlimited for the build:

```
PS> aws ec2 modify-instance-credit-specification --instan
# ...build...
PS> aws ec2 modify-instance-credit-specification --instan
```

4.5 Read the admin credentials / operate the server

```
ssh -i ~/.ssh/reem-key.pem ubuntu@34.199.20.93 "grep SEED
ssh -i ~/.ssh/reem-key.pem ubuntu@34.199.20.93 "cd ~/Loca
```

Admin map: <https://34.199.20.93/admin/> (accept the self-signed cert warning).

4.6 If your home IP changes

SSH will stop connecting (the group only allows your old IP). Update it:

```
PS> aws ec2 authorize-security-group-ingress --group-id <
PS> aws ec2 revoke-security-group-ingress --group-id <s
```

4.7 Tear down (stop all AWS charges)

```
PS> aws ec2 terminate-instances --instance-ids <id>
PS> aws ec2 release-address --allocation-id <alloc>
PS> aws ec2 delete-security-group --group-id <sg>
PS> aws ec2 delete-key-pair --key-name reem-key
```

5. Git workflow

Work happens on `feature/ios-client-admin-map`; `main` is the base. To merge, open a PR: <https://github.com/ReemEl-mohandes/LocationTracker/pull/new/feature/ios-client-admin-map>. Before committing, the repo is scanned so no `.env`, TLS key, SSH key, or admin password is ever staged; `.gitignore` already excludes `.env`, `nginx/certs/*`, iOS build output, and Postman files.

6. Fast troubleshooting

Symptom	Cause / fix
<code>xtool ... Failed to obtain Swift version</code>	Not a login shell — use <code>bash -lc</code>
<code>xtool dev build --ipa</code> fails on zip	Stock Ubuntu lacks zip; use <code>./package.sh</code> (<code>mkipa.py</code>)
App installs but won't open	Crash at launch — read Analytics Data crash log, or bisect features
App opens, can't connect to server	Wrong URL, or cert fingerprint not in <code>AppConfig</code>
Admin map shows a user offline unexpectedly	>5 min since server last received a point (offline / app killed)
Server build crawls on EC2	t3.micro on standard credits — set unlimited for the build
SSH Connection timed out to EC2	Home IP changed — re-authorize port 22 (4.6)
API won't start	<code>JWT_SIGNING_KEY</code> < 32 bytes, or DB not healthy yet

429 from the API

Rate limiter (login 5/min, register 3/hr) —
wait or `docker compose restart`
`api` locally

Keep this current: when a version, path, or command changes, update it here so a fresh setup never guesses.